SOFTWARE REQUIREMENTS SPECIFICATION

15 Real-World Full Stack Internship Projects

Client-Style SRS Collection

Version 1.0

September 2026

Purpose: Client-ready requirements for interns to understand the business problem, scope, workflows, data, APIs, testing and final acceptance before development.

How Interns Must Use This SRS

- Read the complete project specification before writing code.
- Convert requirements into user stories, use cases and wireframes.
- Prepare an ER diagram and database schema before implementing backend APIs.
- Define REST APIs and request/response formats before frontend-backend integration.
- Build the MVP first; advanced features must not delay the core workflow.
- Implement authorization on the backend for every protected operation.
- Test normal, invalid, boundary and role-specific cases.
- Use Git with feature branches and meaningful commits.
- Submit source code, README, ER diagram, API documentation, test cases, screenshots and deployment/local setup instructions.

Common Non-Functional Requirements

- Responsive UI for desktop, tablet and mobile.
- Secure password hashing; never store plain-text passwords.
- Server-side validation for important fields.
- Clear success/error messages and useful HTTP status codes.
- Role-based authorization for protected APIs.
- Database consistency for payments, stock and status changes.
- Search, filtering and pagination for large lists.
- Audit/history for important status and financial changes.
- Environment variables for secrets and API keys.
- Clean folder structure, reusable components and documented setup.
- Basic accessibility: labels, keyboard-friendly forms and useful validation feedback.

. Wedding & Event Planner Web Application

Project Type	Full Stack Web Application
Real-world Client	Local wedding/event planner
Primary Objective	Manage event clients, requirements, budgets, vendors, tasks, payments and event execution.
User Roles	Admin/Planner; Staff; Vendor; Client

Problem Statement

The client needs a single digital system to replace fragmented manual processes such as notebooks, spreadsheets, phone/WhatsApp coordination or separate records. The application must provide one reliable source of truth for the project's operational workflow.

Functional Modules

Module	Requirements
Client Management	Client profile, contacts, requirements and event history.
Event Management	Event type, dates, venue, guest count, schedule and package.
Quotation & Budget	Create quotations, planned expenses and approved budget.
Vendor Management	Decorators, caterers, photographers, DJs, venues and payment terms.
Task Management	Event tasks, assigned staff, deadlines and completion status.
Payment Milestones	Advance, milestone payments, vendor payments and final payment.
Documents	Upload contracts, quotations and event documents.
Client Portal	View approved scope, schedule, payment status and selected vendors.
Notifications	Task deadlines, payment reminders and event reminders.
Reports	Event profitability, budgets, vendor costs and pending payments.

Core Business Flow

Client Management → Event Management → Quotation & Budget → Vendor Management → Task Management → Payment Milestones

Suggested Database Entities

Users, Clients, Events, EventTasks, Vendors, Quotations, Budgets, Contracts, Payments, Notifications

Business Rules

- An event should have one approved budget version.
- Vendor assignments should be linked to a specific event.
- Payment milestones must retain their due dates and payment status.
- Only authorized users can modify approved quotations.

Key Screens

- Login/Register
- Role-specific Dashboard
- Main Records List + Search/Filters
- Create/Edit Form
- Record Details + History
- Transactions/Payments where applicable
- Reports/Analytics
- Profile/Settings/Notifications

API Expectations

Design REST APIs around the actual entities and workflows. Provide authenticated CRUD/search endpoints for core entities, workflow/status endpoints, transaction endpoints where applicable, dashboard/report endpoints, and backend role/permission checks.

Typical pattern: POST /api/auth/login, GET /api/<resource>, POST /api/<resource>, GET /api/<resource>/:id, PUT/PATCH /api/<resource>/:id, DELETE/ARCHIVE /api/<resource>/:id.

MVP Requirements

- Authentication and role-based authorization
- Main customer/user records
- Primary business transaction/workflow
- Search and filtering
- Payment/financial tracking where relevant
- Dashboard with key metrics
- Responsive frontend
- Backend validation and error handling
- Database persistence
- Basic testing and documentation

Advanced / Phase 2 Features

- Client approval portal
- Vendor payment tracking
- Budget variance analytics

End-to-End Acceptance Scenario

A couple books a wedding package. The planner creates the event, prepares a quotation, adds vendors and tasks, records the advance, tracks preparations and closes the event after final settlement.

Acceptance Criteria

- All core user roles can authenticate and see only authorized functions.
- Main business records can be created, viewed, edited and searched.
- The primary workflow can be completed end-to-end without manual database edits.
- Financial calculations, if present, are consistent and testable.
- Important status changes are stored with dates/times and user information where appropriate.
- Invalid inputs produce understandable validation messages.
- Responsive UI works on common desktop and mobile screen sizes.
- No critical frontend, API or database errors remain in the final demonstration.
- README, setup instructions, ER diagram, API documentation and test cases are submitted.
- The project can be demonstrated using realistic sample data.

Recommended 4-Week Internship Plan

Week	Deliverables
Week 1	Requirement analysis, user stories, wireframes, ER diagram, database schema, project setup and authentication.
Week 2	Core modules, database models, backend APIs and primary frontend screens.
Week 3	Business workflow, payments/transactions, dashboard, permissions and frontend-backend integration.
Week 4	Testing, validation, responsive design, bug fixing, documentation, deployment and final client demo.